GraphIoT: Accurate IoT Identification based on Heterogeneous Graph

Linna Fan*†, Lin He†‡§, Xiaoqing Sun†, Enhuan Dong†‡§,
Jiahai Yang†‡§, Zhiliang Wang†‡§, Jinlei Lin† and Guanglei Song†‡
* College of Information and Communication, National University of Defense Technology, Wuhan, China
† Institute for Network Sciences and Cyberspace, BNRist, Tsinghua University, Beijing, China
‡ Zhongguancun Laboratory, Beijing, China
§ Quan Cheng Laboratory, Jinan, Shandong, China

2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS) | 979-8-3503-9973-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/IWQOS57198.2023.10188710

***Abstract*—IoT devices deployed on campus and enterprise networks facilitate people's lives and work. However, these devices also bring serious network asset management and security management problems. IoT device identification is the premise to solve these problems. Although current IoT identification methods can identify devices with relatively high accuracy in ideal environments, it is difficult to accurately identify devices in real-world complex environments (e.g., campus networks, enterprise networks). Therefore, we propose to use exact features. To solve the problem of different dimensions of exact features, we creatively model the IoT identification problem as a heterogeneous graph representation learning problem and design a new representation learning algorithm. We are the first to propose an approach to accurately identify IoT devices in real-world complex environments and solve this problem through heterogeneous graphs. The evaluation shows that *GraphIoT*'s macro F1 is on average 13.58% and 12.77% higher than the other methods on two public datasets.**

***Index Terms*—network management, device identification, IoT, heterogeneous graph**

I. Introduction

Although IoT devices are widely used in various areas of work and life, a large number of IoT devices also increase the difficulty of network asset management and security management in these networks [1].

In the past few years, a considerable amount of research [2]–[18] has been conducted on IoT device identification, yet they suffer from two shortcomings in practical applications. (1) Identification accuracy. The existing work is mainly designed for ideal environments, ignoring the interference traffic in real environments, which can lead to a decrease in identification accuracy. (2) High overhead caused by labeled data. Existing methods usually require a large amount of labeled data to train models, meaning more IoT devices need to be manually labeled. Fig. 1 shows the confusion matrix of the three groups of devices, which is used to describe the performance of the classifier. Even using 70% of the labeled data, these methods still have difficulty distinguishing different devices.

Therefore, we focus on accurate IoT device identification in real-world complex environments using as little human-labeled data as possible. Real-world complex environments are those environments that may contain interference traffic, causing the normal operating traffic of the device to be affected. This paper proposes *GraphIoT*, whose goal is to accurately identify IoT devices in real-world complex environments using less human annotation. The core idea of *GraphIoT* is to extract highly discriminative exact features from traffic and train a model using labels from a small number of devices, and then use that model to identify IoT devices. However, there are two challenges in realizing such a model. *1) Different feature dimensions brought about by exact features* make traditional machine learning models inapplicable. *2) Using only a small amount of labeled data* increases the difficulty of identifying devices from the same vendor in real environments.

GraphIoT uses the following innovative designs to address these challenges. *First, GraphIoT uses graphs for modeling to adapt to different feature dimensions.* The use of accurate features faces the problem of different feature dimensions. To solve this problem, we use a heterogeneous graph and creatively consider IoT device instances and features as nodes in the graph to represent the relationship between IoT device instances and features (§ II-B). *Second, GraphIoT leverages graph representation learning for device classification.* Using less labeled data to train a model that can accurately classify devices is more challenging because IoT devices have different running stages, resulting in different traffic characteristics. To address this issue, we design a heterogeneous graph representation learning algorithm (§ II-C).

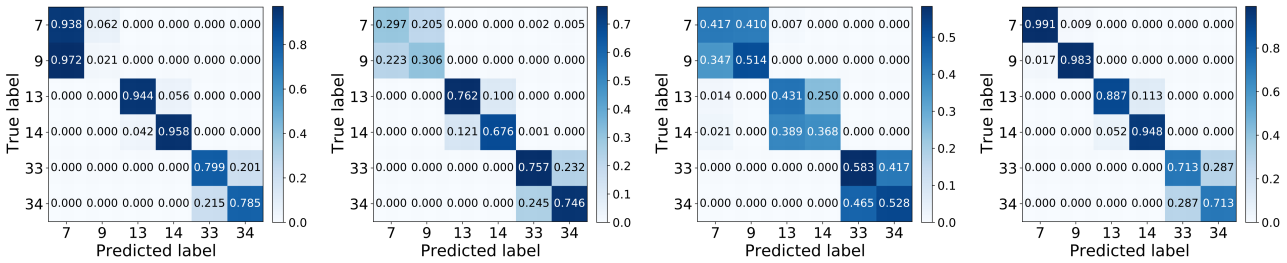
We test *GraphIoT* on two public datasets. The experimental results show that *GraphIoT* outperforms other state-of-the-art methods at various label percentages. *GraphIoT*'s $macro\ F1$ is on average 13.58% and 12.77% higher than the other methods on the two datasets. Especially for some hard-to-distinguish devices, *GraphIoT* can accurately distinguish them using only 20% of the labels, while other methods can hardly achieve this even using 70% of the labels.

II. *GraphIoT*

This section introduces the design of *GraphIoT*, including Feature Extraction, Heterogeneous Graph Construction and Heterogeneous Graph Representation Learning, shown in Fig. 2.

979-8-3503-9973-8/23/$31.00 ©2023 IEEE.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:56:28 UTC from IEEE Xplore. Restrictions apply.



(a) Sivanathan et al. [6] (b) Shahid et al. [8] (c) AuDI [14] (d) SemiModel [15]

Fig. 1. The confusion matrices of different methods trained on Yourthings dataset.

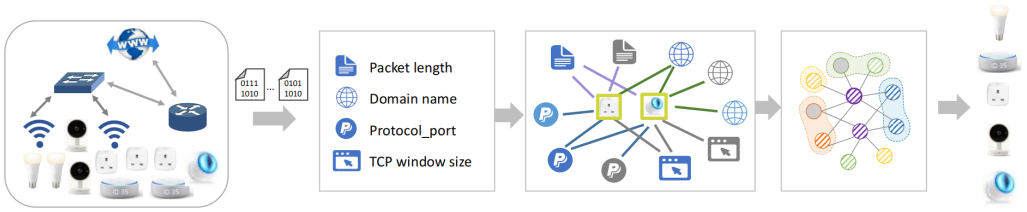


Fig. 2. High-level overview of *GraphIoT*.

A. Feature Extraction

GraphIoT extracts stable and discriminable features from the traffic for subsequent graph construction. Here we only consider features extracted from packets sent by IoT devices and select relatively stable protocol features, because features such as the port number and domain names in packets sent by devices are mainly related to device type and function and are less affected by interference traffic, while time interval and traffic features fluctuate with changes in a network environment. To find important features, we extract the 219-dimensional statistical features used by SemiModel [15]. We then calculate the importance of these features using random forest feature selection [19]. Ultimately, we chose packet length, protocol_port, domain name, and TCP window size. To distinguish the ports of TCP and UDP, we combine protocol and port to form protocol_port. We add 100000 to the UDP port number to distinguish it from the TCP port number.

B. Heterogeneous Graph Construction

In this step, *GraphIoT* establishes the relationships between IoT instances and features through a graph, which can be unconstrained by feature dimensions. Since there are four types of features, the relationship between IoT instances and features includes IoT-use-Packet length, Packet length-used_by-IoT, IoT-request-Domain, Domain-requested_by-IoT, IoT-use-Protocol_port, Protocol_port-used_by-IoT, IoT-use-TCP window size and TCP window size-used_by-IoT.

IoT-use-Packet length represents the lengths of the TCP packets and UDP packets sent by an IoT device in the time window can be expressed as IoT-use-Packet length. If the packet lengths of instance IoT_i's packets contains l_i, there is a link from IoT_i to l_i. Packet length-used_by-IoT represents the inverse relationship of IoT-use-Packet length. For reasons of space, we do not illustrate other relationships.

Based on the different relationships between instances and features above, we can construct edges between instances and features to form a graph.

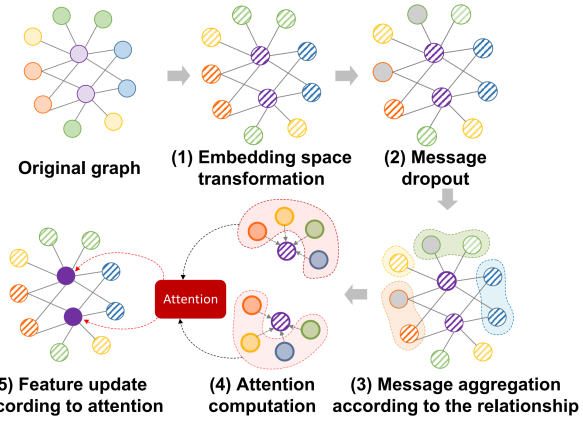


Fig. 3. Steps of Heterogeneous Graph Representation Learning algorithm.

C. Heterogeneous Graph Representation Learning

GraphIoT learns embeddings of nodes through heterogeneous graph representation learning, using a small number of labeled IoT instances. The embeddings of IoT instances are then used to further obtain their prediction classes. To this end, we design a heterogeneous graph representation learning algorithm that can take into account the contributions of different types of features during the update process. The algorithm is divided into five steps shown in Fig. 3.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:56:28 UTC from IEEE Xplore. Restrictions apply.

Step 1. Embedding space transformation

First, we need to transform the initial embeddings of nodes into embeddings to generate subsequent messages for propagation. We apply a relation-specific linear transformation for each relationship by projecting embedding $\mathbf{h_u}$ of node u into the same latent feature space. The message generated can be expressed as Equation (1):

$$\mathbf{h}_u^{(l+1)} = \mathbf{W}_r^{(l)} \mathbf{h}_u^{(l)}, \tag{1}$$

where $\mathbf{h}_u^{(l+1)}$ is the transformed embedding of node u in layer $l+1$. $\mathbf{h}_u^{(l)}$ is the embedding of u in layer l. r is a relationship in § II-B. $\mathbf{W}_r^{(l)}$ is the weight matrix of r in layer l. The same relationship r share a common weight matrix $\mathbf{W}_r^{(l)}$.

Step 2. Message dropout

After the embedding space transformation, each node sends its embedding to the endpoints along the edges. Embedding that propagates on edge is called a message. Each endpoint randomly selects some of the messages sent to it for subsequent message aggregation. We call this process message dropout. To finish this, we generate a random number $rand_{(u,v)}^{(l+1)} \sim Bernoulli(p)$ for each edge (u, v). Then message $\mathbf{m}_{(u,v)}^{(l+1)}$ after dropout is:

$$\mathbf{m}_{(u,v)}^{(l+1)'} = rand_{(u,v)}^{(l+1)} \mathbf{h}_u^{(l+1)}. \tag{2}$$

Step 3. Message aggregation according to the relationship

After message dropout, each destination node v aggregates messages for each relationship r and forms the aggregated message $\mathbf{m_aggr}_{v,r}^{(l+1)}$:

$$\mathbf{m_aggr}_{v,r}^{(l+1)} = \frac{1}{|N_u|} \sum_{u \in Neigh_v, R_map(u,v)=r} \mathbf{m}_{(u,v)}^{(l+1)'}, \tag{3}$$

where $Neigh_v$ stands for the neighbors of v and $R_map(u, v)$ can map the edge (u, v) to a relationship r. $r \in R$. R is the relationship set in § II-B. $|N_u|$ is the count of all u used in message aggregation.

Step 4. Attention computation

To get better performance, we apply the attention mechanism to IoT nodes when aggregating $\mathbf{m_aggr}_{v,r}^{(l+1)}$ with $r \in R$. The process is:

$$e_{v,r}^{(l+1)} = \mathbf{w}_{att} \cdot tanh(\mathbf{W}_{att}^{(l+1)} \mathbf{m_aggr}_{v,r}^{(l+1)}). \tag{4}$$

$\mathbf{W}_{att}^{(l+1)} \in R^{d_2^{(l)} \times d_1^{(l)}}$ can transform the dimension $d_1^{(l)}$ of $\mathbf{m_aggr}_{v,r}^{(l+1)}$ into dimension $d_2^{(l)}$. $\mathbf{w}_{att} \in R^{1 \times d_2^{(l)}}$.

$$e_r^{(l+1)} = \frac{1}{|r_{dst}|} \sum_{i \in r_dst} e_{i,r}^{(l+1)}, \tag{5}$$

where r_{dst} is the endpoint set of edges belonging to relationship r. After that, the normalized value of e can be computed through formula (6):

$$\alpha_r^{(l+1)} = \frac{exp(e_r^{(l+1)})}{\sum_{i \in R} exp(e_i^{(l+1)})}, \tag{6}$$

Step 5. Feature update according to attention

The last step is to aggregate $\mathbf{m_aggr}_{v,r}^{(l+1)}$ of different relationship r for each node v according to attention. The formula is:

$$\mathbf{h}_v^{(l+1)} = \begin{cases} \sum_{r \in R} \alpha_r^{(l+1)} \mathbf{m_aggr}_{v,r}^{(l+1)}, l+1 = L \\ LeakyRelu(\sum_{r \in R} \alpha_r^{(l+1)} \mathbf{m_aggr}_{v,r}^{(l+1)}), l+1 \neq L \end{cases} \tag{7}$$

L stands for the maximum number of layers. $\mathbf{h}_v^{(l+1)}$ can be used to update the embedding of node v. These five steps are repeated to update node embedding in the next layer until the last layer.

Training. We only use a small fraction of labeled IoT instances' data to optimize the model weights by minimizing the cross-entropy loss, which is a commonly used loss function for classification tasks. The loss can be expressed as:

$$\mathbb{L} = -\sum_{v \in V_{Label}} \sum_{c=1}^{C} y_v[c] \cdot log \mathbf{h}_v[c], \tag{8}$$

where V_{Label} is the set of nodes that have labels. C is the class number. y_v is the one-hot label vector of the true label. $\mathbf{h}_v$ is the output logits of the last layer of node v. By minimizing $\mathbb{L}$ using backpropagation and gradient descent, we can train a classifier to predict labels of other nodes. The predicted class is the index of the maximum value of the final embedding.

III. Evaluation

We compare *GraphIoT* with other IoT device identification efforts on Yourthings [20] and UNSW datasets [6]. Baseline methods include Sivanathan et al. [6], Shahid et al. [8], AuDI [14], and SemiModel [15].

Fig. 4 and Fig. 5 show the performance of these methods under different labeling percentages. The $macro\ F1$ of *GraphIoT* is the highest at all label percentages on both datasets and it is 13.58% and 12.77% higher than other methods on average on two datasets respectively. The slight drop in $macro\ F1$ for Sivanathan et al. [6] with 40% of the data labeled on UNSW dataset is because the device Withings Baby Monitor has only one instance in the test set, which was misclassified.

Although other methods, such as Sivanathan et al. [6] and SemiModel [15], also have high $macro\ F1$ when the label percentage is high on Yourthings dataset, this does not mean that they provide accurate identification of the three groups of devices because other easily distinguishable devices can pull up the overall $macro\ F1$. Therefore, we focus more on the three groups of devices that are not easily distinguishable.

Fig. 6 compares the average accuracy of these methods for these three groups of devices at different label percentages. The average accuracy of *GraphIoT* is the highest at each label ratio. Under the 70% labeling ratio, the average accuracies of SemiModel and Sivanathan et al. [6] are only 74.07% and 87.25%, respectively, which are much lower than 98.5% of *GraphIoT*. Further, *GraphIoT* has an average accuracy

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:56:28 UTC from IEEE Xplore. Restrictions apply.

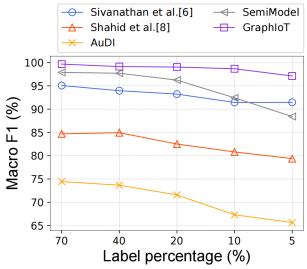


Fig. 4. Performance comparison on Yourthings dataset.

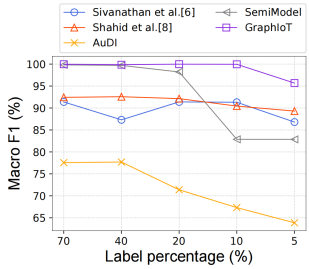


Fig. 5. Performance comparison on UNSW dataset.

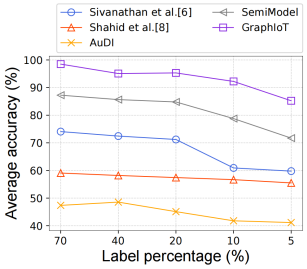


Fig. 6. Average accuracy comparison of the three groups of devices.

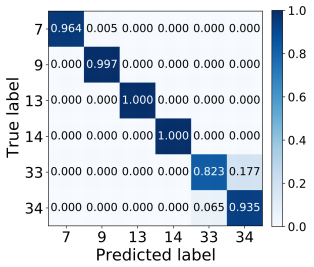


Fig. 7. The confusion matrix of *GraphIoT* on Yourthings dataset.

of 95.31% for the three groups of devices even at a 20% label percentage, which is higher than the performance of other methods at a 70% label percentage. Also, we plot the confusion matrix of *GraphIoT* at the 20% label percentage, as shown in Fig. 7. Compared with the confusion matrix of other methods at the 70% label percentage in Fig. 1, *GraphIoT* can best identify these three groups of not easily distinguishable devices even at a lower label percentage.

IV. Conclusion

In this paper, we analyze the problems of current methods in accurately identifying IoT devices. To use accurate features, we model the IoT identification problem as a heterogeneous information network. Further, to learn the rich semantics in nodes and links, we propose a novel *GraphIoT* algorithm to learn the representation of nodes. Experimental results show that the $macro\ F1$ of *GraphIoT* is on average 13.58% and 12.77% higher than other methods.

V. Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62172251), and Beijing Natural Science Foundation (No. 4222026). Jiahai Yang and Lin He are the corresponding authors of this paper.

References

[1] L. Fan, L. He, E. Dong, J. Yang, C. Li, J. Lin, and Z. Wang, "Evoiot: An evolutionary iot and non-iot classification model in open environments," *Computer Networks*, vol. 219, pp. 1–12, 2022.

[2] B. Bezawada, M. Bachani, J. Peterson, H. Shirazi, I. Ray, and I. Ray, "Iotsense: Behavioral fingerprinting of IoT devices," *arXiv preprint arXiv:1804.03852*, 2018.

[3] A. Bremler-Barr, H. Levy, and Z. Yakhini, "IoT or Not: Identifying IoT devices in a short time scale," in *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2020, pp. 1–9.

[4] S. A. Hamad, W. E. Zhang, Q. Z. Sheng, and S. Nepal, "IoT device identification via network-flow based fingerprinting and learning," in *2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*. IEEE, 2019, pp. 103–111.

[5] N. Msadek, R. Soua, and T. Engel, "IoT device fingerprinting: Machine learning based encrypted traffic analysis," in *2019 IEEE wireless communications and networking conference (WCNC)*. IEEE, 2019, pp. 1–8.

[6] A. Sivanathan, H. H. Gharakheili, F. Loi, A. Radford, C. Wijenayake, A. Vishwanath, and V. Sivaraman, "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Transactions on Mobile Computing*, vol. 18, no. 8, pp. 1745–1759, 2018.

[7] M. Skowron, A. Janicki, and W. Mazurczyk, "Traffic fingerprinting attacks on internet of things using machine learning," *IEEE Access*, vol. 8, pp. 20 386–20 400, 2020.

[8] M. R. Shahid, G. Blanc, Z. Zhang, and H. Debar, "IoT devices recognition through network traffic analysis," in *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 2018, pp. 5187–5192.

[9] A. Aksoy and M. H. Gunes, "Automated IoT device identification using network traffic," in *ICC 2019-2019 IEEE International Conference on Communications (ICC)*. IEEE, 2019, pp. 1–7.

[10] J. Bao, B. Hamdaoui, and W.-K. Wong, "IoT device type identification using hybrid deep learning approach for increased IoT security," in *2020 International Wireless Communications and Mobile Computing (IWCMC)*. IEEE, 2020, pp. 565–570.

[11] M. Miettinen, S. Marchal, I. Hafeez, N. Asokan, A.-R. Sadeghi, and S. Tarkoma, "Iot sentinel: Automated device-type identification for security enforcement in IoT," in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2017, pp. 2177–2184.

[12] Y. Meidan, M. Bohadana, A. Shabtai, M. Ochoa, N. O. Tippenhauer, J. D. Guarnizo, and Y. Elovici, "Detection of unauthorized IoT devices using machine learning techniques," *arXiv preprint arXiv:1709.04647*, 2017.

[13] Y. Song, Q. Huang, J. Yang, M. Fan, A. Hu, and Y. Jiang, "IoT device fingerprinting for relieving pressure in the access control," in *Proceedings of the ACM Turing Celebration Conference-China*, 2019, pp. 1–8.

[14] S. Marchal, M. Miettinen, T. D. Nguyen, A.-R. Sadeghi, and N. Asokan, "AuDI: Toward autonomous IoT device-type identification using periodic communication," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1402–1412, 2019.

[15] L. Fan, S. Zhang, Y. Wu, Z. Wang, C. Duan, J. Li, and J. Yang, "An IoT device identification method based on semi-supervised learning," in *2020 16th International Conference on Network and Service Management (CNSM)*. IEEE, 2020, pp. 1–7.

[16] J. Ortiz, C. Crawford, and F. Le, "Devicemien: network device behavior modeling for identifying unknown IoT devices," in *Proceedings of the International Conference on Internet of Things Design and Implementation*, 2019, pp. 106–117.

[17] A. Sivanathan, H. H. Gharakheili, and V. Sivaraman, "Inferring IoT device types from network behavior using unsupervised clustering," in *2019 IEEE 44th Conference on Local Computer Networks (LCN)*. IEEE, 2019, pp. 230–233.

[18] L. Fan, L. He, Y. Wu, S. Zhang, Z. Wang, J. Li, J. Yang, C. Xiang, and X. Ma, "Autoiot: Automatically updated iot device identification with semi-supervised learning," *IEEE Transactions on Mobile Computing*, 2022.

[19] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[20] O. Alrawi, C. Lever, M. Antonakakis, and F. Monrose, "Sok: Security evaluation of home-based IoT deployments," in *2019 IEEE symposium on security and privacy (sp)*. IEEE, 2019, pp. 1362–1380.

Authorized licensed use limited to: University of Missouri Libraries. Downloaded on August 13,2026 at 05:56:28 UTC from IEEE Xplore. Restrictions apply.